

M Ű E G Y E T E M 1 7 8 2

BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Elosztott adatgyűjtő rendszerek

Intelligens beágyazott rendszerek laboratórium

(BMEVIMIMA21)

Készítette:

Pál Patrik, Gavallér Dávid

AIQTQW, ES02RO

BUDAPEST, 2026. október 17.

Tartalomjegyzék

1. Bevezetés **2**

2. Mérési feladatok **3**

1. feladat 3
1.1. feladat 3
1.2. feladat 4
1.3. feladat 5
1.4. feladat 8
1.5. feladat 9
1.6. feladat 11
1.7. feladat 13
2. feladat 15
2.1. feladat 15
2.2. feladat 16
2.3. feladat 17
2.4. feladat 18
2.5. feladat 19
2.6. feladat 19

1. Bevezetés

A mérés célja

A mérés célja az elosztott rendszerű hálózatok és azok problémáinak megismerése gyakorlati példákon keresztül.

Az első fő feladatrészben két mitmótot felhasználva vizsgáljuk a szenzor és a gateway által rögzített jelek szinkronizációs problémáit. A feladat során a két mérőegység által rögzített jelek időbeli összehangolását végezzük el időbélyeg-transzformáció segítségével, majd a gateway mintavételi időpontjaiban interpolációval becsüljük a szenzorértékeket.

Ezt követően a második fő feladatrészben Fourier-együtthatókból állítjuk vissza a szenzor jelét és vizsgáljuk a rezonátoros megfigyelő működését.

A méréshez használt eszközök

- Mitmótok
- Analóg interface kártya
- Oszcilloszkóp
- Számítógép
- Függvénygenerátor

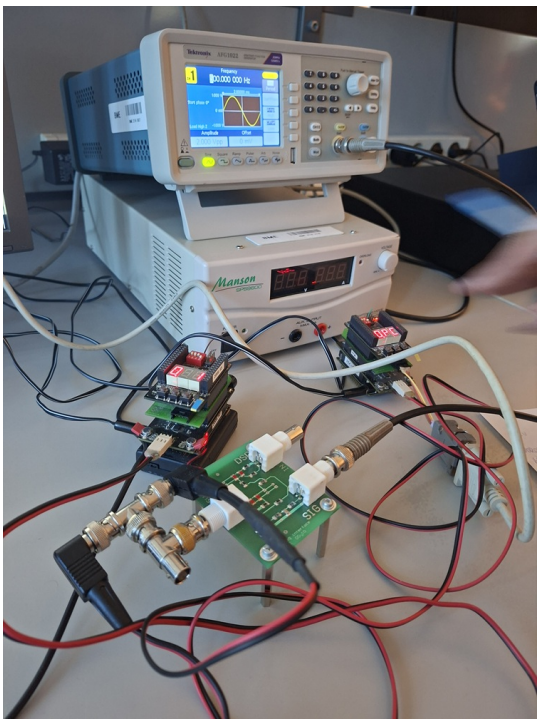
2. Mérési feladatok

1. feladat

Az első feladat célja a különálló mérőegységek által rögzített adatok időbeli összehangolása. A feladat során először beolvassuk és ábrázoljuk a szenzor, az adatgyűjtő, valamint a szinkronizációs mérések nyers adatait. A szinkronizációs pontokra illesztett lineáris időbélyeg-transzformáció segítségével az adatgyűjtő időskáláját a szenzor időskálájára alakítjuk, majd megvizsgáljuk az így kapott jelek egyezését. Az összehangolt adatokból lineáris interpolációval becsüljük a szenzorértékeket az adatgyűjtő mintavételi időpontjaiban. Az elemzést 50 [Hz]-es, 500 [Hz]-es, valamint változó amplitúdójú mérésre is elvégeztük.

1.1. feladat

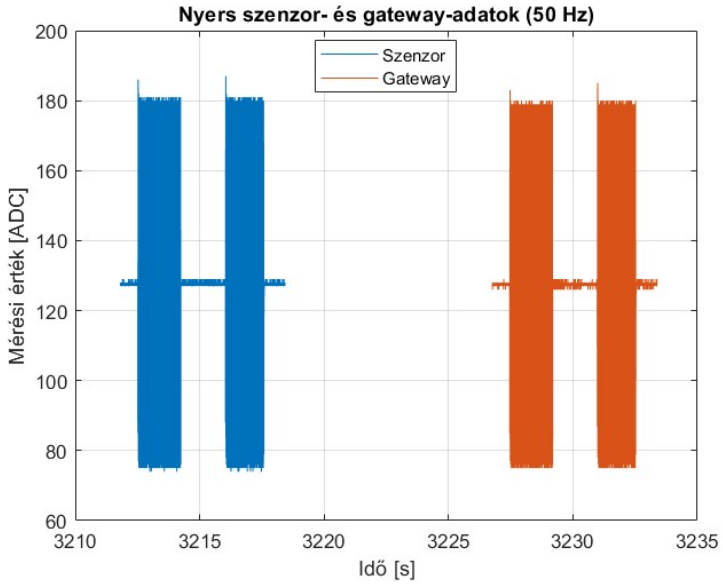
Az első feladat első pontja a mérési összeállítás elkészítése volt. Külső jelként egy függvénygenerátort használtunk, amit az analóg interface kártya Low Pass Filter bemenetére kötöttünk. Az LPS kimenetét az ellenállásokon keresztül a szenzor és a gateway bemenetére kapcsoltunk, ide csatlakoztatva egy oszcilloszkópot is, a jel valós idejű ellenőrzésének céljából. A gateway-t végül a számítógéphez csatlakozattuk, hogy a méréseket tudjuk rögzíteni és kiértékelni. A DIP kapcsolókat a mérési útmutatóban leírt módon beállítottuk, majd a jelet megvizsgáltuk az oszcilloszkópon és matlabban is. Az összeállítás az alábbi 1. ábra-n látható:



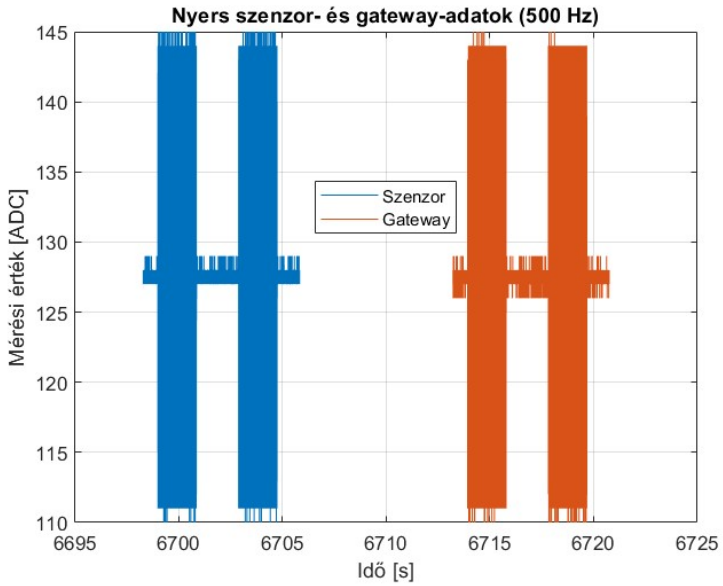
1. ábra. A mérési elrendezés.

1.2. feladat

A következő feladat a mérési regisztrátum készítése volt. A mérés rögzítéséhez a mérési jegyzőkönyvben leírt programot használtuk, majd a `preprocessSampleFiles.m` script futtáatása után kapott, egyszerűen feldolgozható fájlokat vizsgáltuk. A mérést elvégeztük 50 Hz-es és 500 Hz-es szinuszos jelekre is úgy, hogy mérés közben ki és bekapcsoltuk a jelgenerátort. A jelek amplitúdóját mind a kettő esetben 2 V-ra állítottuk, ezzel kitöltve a mérési tartományt. A mérési eredmények 50 Hz-es és 500 Hz-es jelekre az alábbi 2. ábra-n és 3. ábra-n láthatóak:



2. ábra. Nyers szenzoradatok (50 Hz).

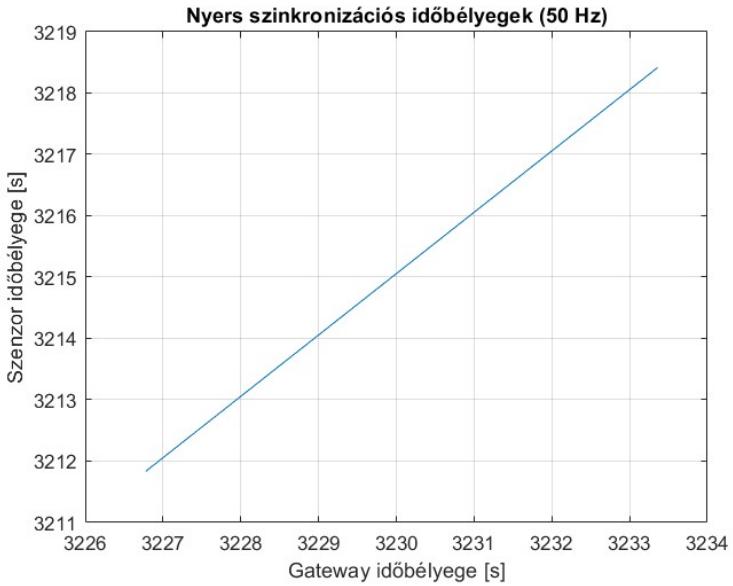


3. ábra. Nyers szenzoradatok (500 Hz).

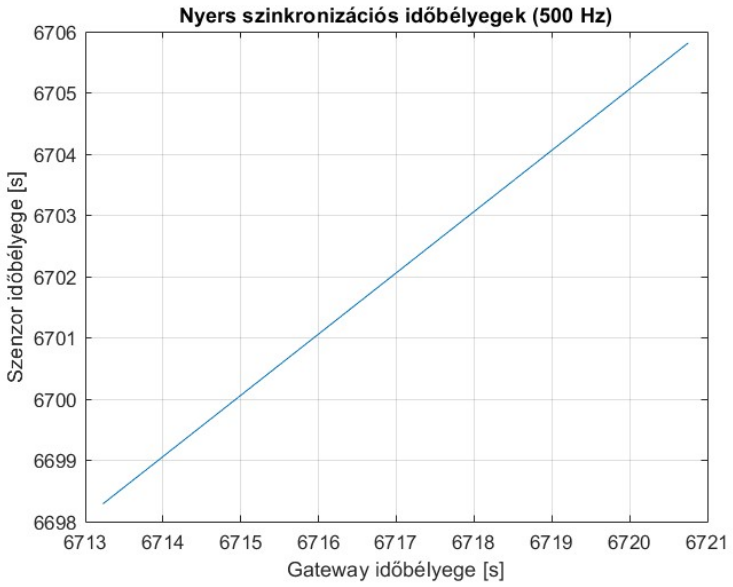
1.3. feladat

A harmadik feladatban az volt a célunk, hogy a szenzor és a gateway által rögzített jeleket időben összehangoljuk. Az időbeli eltérést az eszközöknél kezdeti időeltérés és a két eszköz órajelének eltérése okozhatta. Ennek a megoldására egy olyan lineáris transzformációt kerestünk, ami a gateway időskáláját a szenzor időskálájára alakítja.

Ehhez az `stmpSync` fájlt használtuk fel, amiben szinkronizációs időbélyeg párok szerepelek. Ez azt jelenti, hogy ezek az adatok két eszköz által rögzített, azonos eseményekhez tartozó időbélyegek. A Matlab `polyfit` függvényével ezekre az időbélyegekre lineáris függvényt illesztettünk, ami az alábbi ábrákon látható 50 [Hz]-es és 500 [Hz]-es jelekre:



4. ábra. Szinkronizációs időbélyegek (50 Hz).



5. ábra. Szinkronizációs időbélyegek (500 Hz).

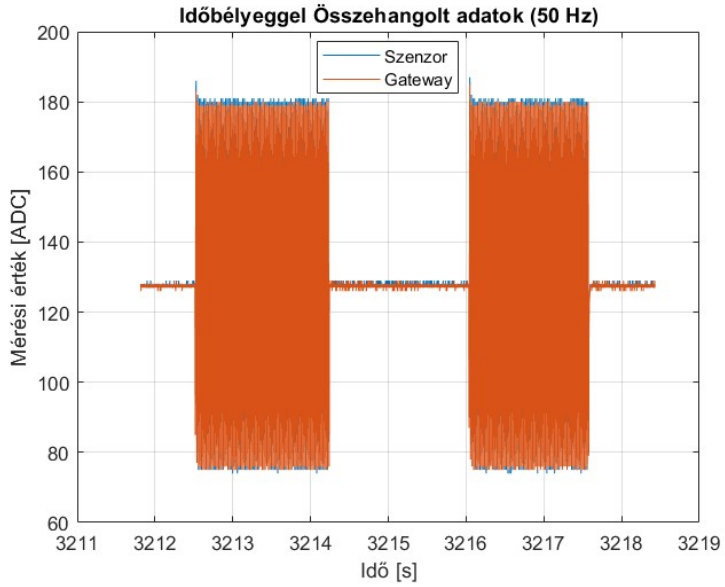
A `polyfit` függvény az illesztett elsőfokú polinom együtthatóit adja vissza. A méréshez tartozó szinkronizációs pontokra az illesztés eredménye közelítőleg

$$\texttt{idobelyegPoli} = [1, -14{,}9512] \tag{1}$$

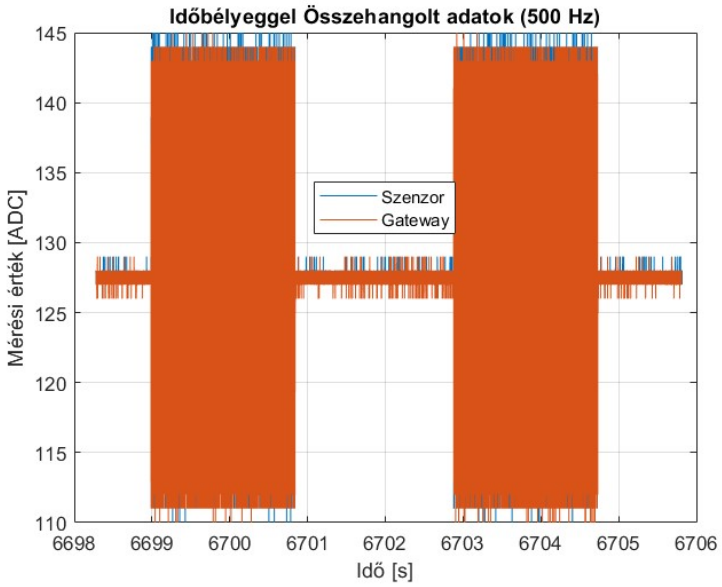
Mivel elsőfokú polinomot illesztettünk, az első érték a meredekség, a második pedig a tengelymetszet. Ezért a gateway időbélyegét a következő összefüggéssel alakítottuk át a szenzor időskálájára:

$$t_{\text{szenzor}} = 1 \cdot t_{\text{gateway}} - 14{,}9512 \tag{2}$$

A tengelymetszet pedig azt a kezdeti időeltérést írja le, amely a két eszköz időbélyegei között fennáll. Az illesztési együtthatókat a `polyval` függvénnyel alkalmaztuk `gwySampX` minden elemére, így kiszámítva az illesztett polinom által becsült, a szenzor időskálájára átszámított időbélyegeket, amik az alábbi ábrákon láthatóak 50 [Hz]-es és 500 [Hz]-es jelek esetén:

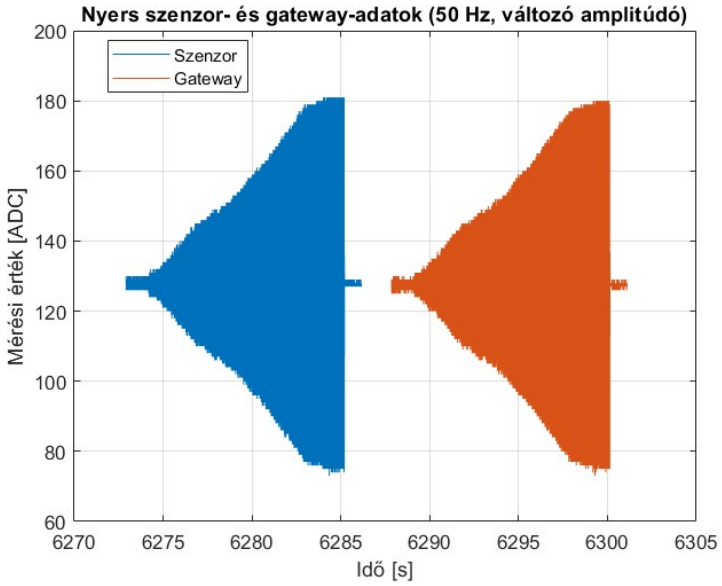


6. ábra. Időbélyeggel összehangolt jelek (50 Hz).

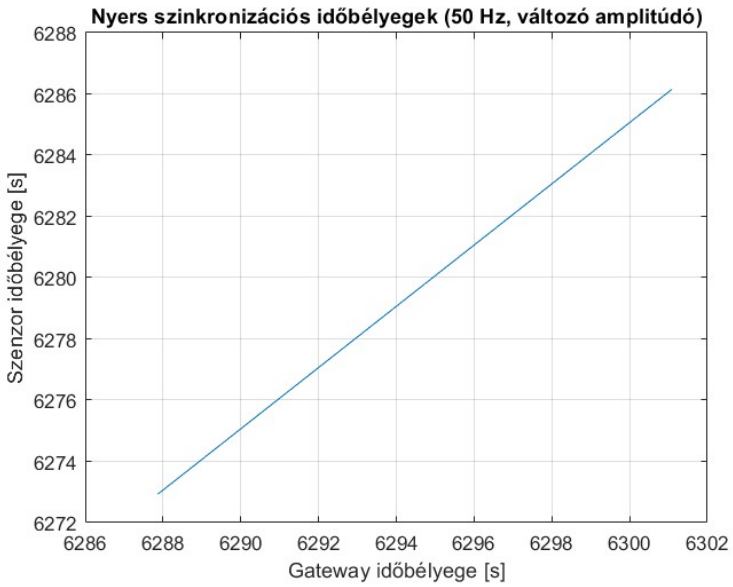


7. ábra. Időbélyeggel összehangolt jelek (500 Hz).

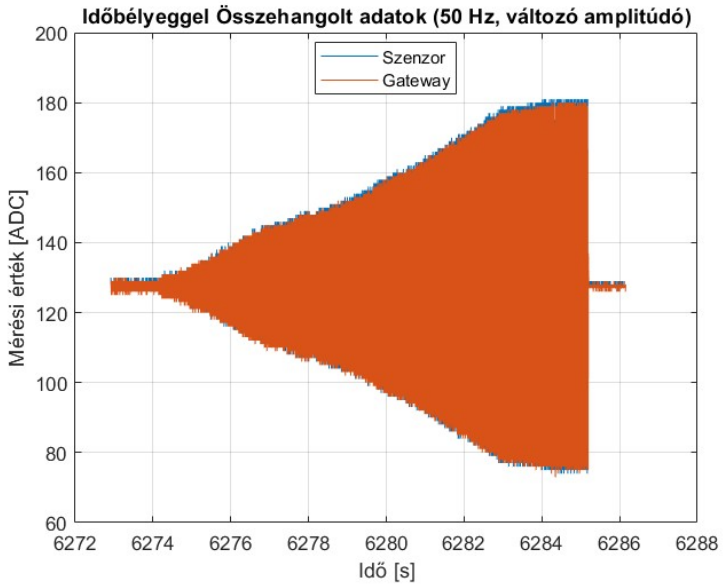
Ezt követően egy mérési regisztrátumot készítettünk, amelyben a jel amplitúdóját az indulástól kezdve folyamatosan növeltük, majd elvégeztük rá a korábbi feladatokban leírt összehangolását. Ez alatt a mérés alatt 50 [Hz]-es jelet generáltunk. Az alábbi ábrákon látható a nyers mérési regisztrátum, a szinkronizációs időbélyegek, valamint az összehangolt jelek:



8. ábra. Nyers szenzor jelek változó amplitúdóval.



9. ábra. Szinkronizációs időbélyegek változó amplitúdóval.



10. ábra. Időbélyeggel összehangolt jelek változó amplitúdóval.

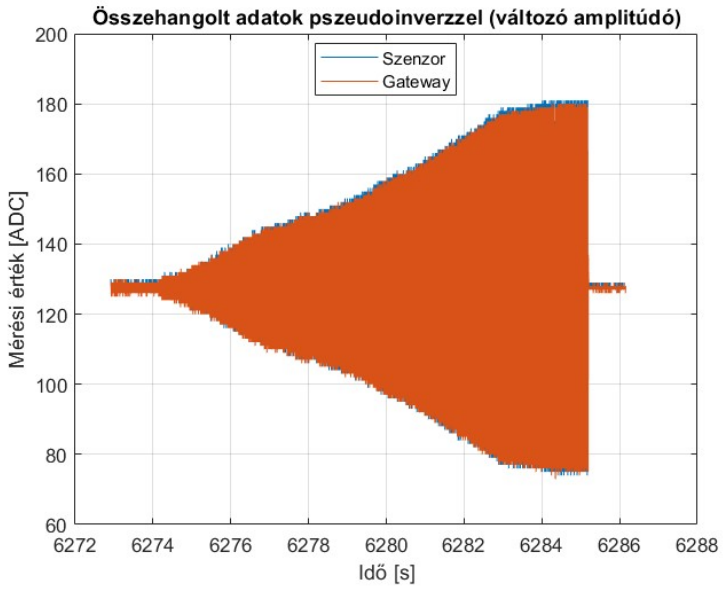
1.4. feladat

A negyedik feladatban az előző feladatban bemutatott időbélyeg-transzformációt végeztük el újra, LS-becslés segítségével. Felépítettük az $\mathbf{X}$ mátrixot, amelynek első oszlopa a gateway szinkronizációs időbélyegeit, második oszlopa pedig egyeseket tartalmazott. Az $\mathbf{y}$ vektor a szenzor szinkronizációs időbélyegeit tartalmazta. A számításhoz a Moore–Penrose-pszeudoinverzt használtuk, amelynek LS-megoldása:

$$p = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{3}$$

A MATLAB beépített függvényei között megtalálható a `pinv` függvény, amellyel ez a megoldás SVD-

alapú módon számítható. A pszeudoinverzzel összehangolt jelek az alábbi ábrán láthatóak:



11. ábra. Pszeudoinverz segítségével összehangolt jelek változó amplitúdóval.

Ezt követően a gateway szinkronizációs időadatait és a gateway mérési időtengelyét 10^6 másodperccel eltoltuk, majd így ismét elvégeztük a feladatot, és vizsgáltuk a mátrix kondicionáltságát. Az első esetben a kondíciószám $1{,}0367 \cdot 10^7$ volt, az időeltolást követően pedig ez az érték $2{,}6496 \cdot 10^{11}$-re növekedett. Az ábrán ennek ellenére nem látható lényeges eltérés, mivel a lineáris transzformáció konstans tagja kompenzálja az időkoordináta közös eltolását. A nagyobb kondíciószámból azonban következik, hogy a becslés érzékenyebbé válik a mérési és kerekítési hibákra.
A probléma megoldására az időadatokat az illesztés előtt központosítottuk. Ehhez a gateway szinkronizációs időadataiból kivontuk azok átlagát, és ugyanezt az eltolást alkalmaztuk a gateway mérési időpontjaira is. Így a transzformáció eredménye nem változik, miközben a mátrix kondicionáltsága javul.

1.5. feladat

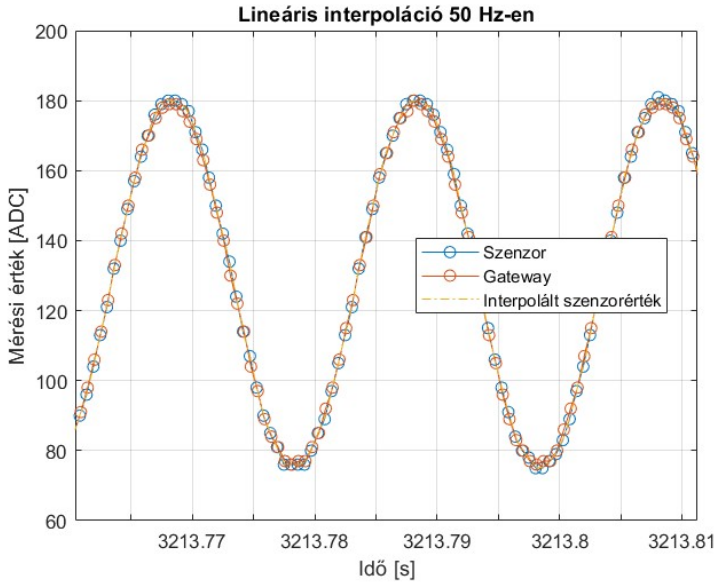
Az ötödik feladatban a szenzor mérési eredményeit a gateway mintavételi időpontjaiban kellett meghatározni lineáris interpoláció segítségével. Először a szinkronizációs pontokból lineáris időbélyegtranszformációt hajtottunk végre, a korábbi feladatokban leírt módszer segítségével. Az `interp1` függvény ezekben az időpontokban a szenzor két szomszédos mintája közötti egyenest használva becsülte meg a szenzor mérési értékét, az alábbi összefüggés szerint:

$$\hat{s}(t) = s(t_i) + \frac{t - t_i}{t_{i+1} - t_i}\big(s(t_{i+1}) - s(t_i)\big), \qquad t_i \le t \le t_{i+1}. \tag{4}$$

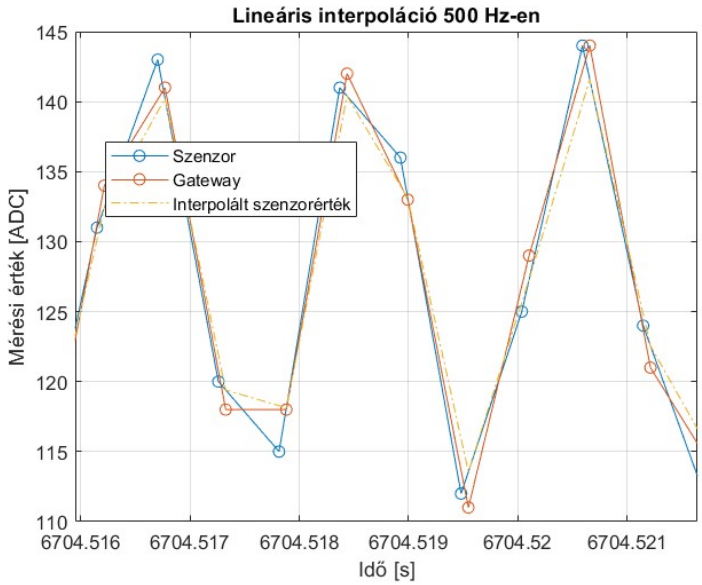
A kapott interpolált szenzorértékeket a gateway ugyanazon időpontokban mért értékeivel hasonlítottuk össze. Az összehasonlításhoz az átlagos négyzetes eltérést is kiszámítottuk:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\mathrm{gateway},i} - \hat{y}_{\mathrm{szenzor},i}\right)^2. \tag{5}$$

A programot 50 [Hz]-es és 500 [Hz]-es mérésre is lefuttattuk. Az 50 [Hz]-es jel esetén az MSE értéke 0.7255, 500 [Hz]-es jel esetén pedig 1.9439 volt. Az 500 [Hz] nagyobb változási sebessége volt a nagyobb hiba oka a második esetben. A mérési eredmények az alábbi ábrákon láthatóak, kellő nagyítással a jobb vizualizáció érdekében:



12. ábra. Lineáris interpoláció 50 Hz-en.



13. ábra. Lineáris interpoláció 500 Hz-en.

1.6. feladat

A hatodik feladatban a szenzor mérési eredményeit felmintavételezéssel sűrítettük annak érdekében, hogy a gateway mintavételi időpontjaiban pontosabban tudjuk meghatározni a szenzorjel értékét. A felmintavételezésre azért volt szükség, mert a két mérőegység nem ugyanabban az időpillanatban rögzítette a mintákat. A feldolgozást az 50 [Hz]-es és az 500 [Hz]-es mérésre is elvégeztük. A szenzorjelet tízszeresen mintavételeztük fel. Ennek során az eredeti szenzorminták közé nullákat illesztettünk, majd az így kapott jelet aluláteresztő szűrővel simítottuk.
Az eredeti szenzorminták közötti átlagos időeltérésből meghatároztuk az eredeti mintavételi frekvenciát:

$$T_s = \text{mean}\left(\text{diff}(t_{\text{szenzor}})\right), \qquad f_s = \frac{1}{T_s}. \tag{6}$$

A tízszeres felmintavételezés után az új mintavételi frekvencia:

$$f_{s,\text{j}} = 10 f_s. \tag{7}$$

A nullákkal kiegészített jelet negyedrendű Butterworth-aluláteresztő szűrővel szűrtük. A szűrő vágási frekvenciáját az eredeti mintavételi frekvencia felének választottuk. A szűrést a MATLAB `filtfilt` függvényével végeztük, amely előre és visszafelé is végrehajtja a szűrést, ezért nem okoz fázistolást. A szűrés után a jelet a felmintavételezési tényezővel skáláztuk, hogy a mérési értékek amplitúdója megfelelő maradjon.

Ezt követően létrehoztuk a felmintavételezett szenzorjelhez tartozó új időtengelyt. Az időpontokat a mérés kezdő időpontjától számítottuk az új, sűrűbb mintavételi frekvenciával. Ezután a korábbi feladatokban ismertetett időbélyeg-transzformációt alkalmaztuk. A szinkronizációs időbélyegpárokra elsőfokú polinomot illesztettünk, majd a `polyval` függvénnyel a gateway minden mintavételi időpontját átszámítottuk a szenzor időskálájára.
A szenzorértékeket ezekben az időpontokban lineáris interpolációval határoztuk meg. Az interpoláció során a keresett időpontot közrefogó két szenzorminta közötti egyenest használtuk:

$$\hat{s}(t) = s(t_i) + \frac{t - t_i}{t_{i+1} - t_i}\left(s(t_{i+1}) - s(t_i)\right), \qquad t_i \le t \le t_{i+1}. \tag{8}$$

elsőfokú polinomot illesztettünk, majd a `polyval` függvénnyel a gateway minden mintavételi időpontját átszámítottuk a szenzor időskálájára.

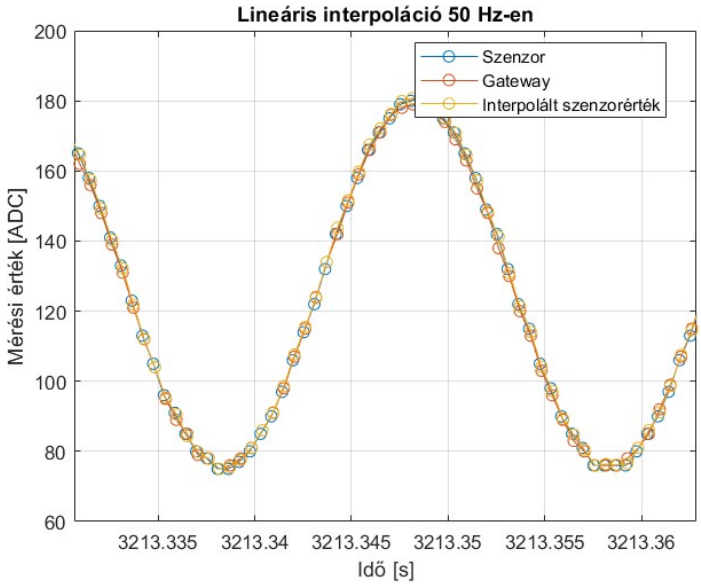
Az interpolációt a MATLAB `interp1` függvényével valósítottuk meg. Az így kapott szenzorértékeket a gateway ugyanazon időpontokban mért értékeivel hasonlítottuk össze. Az összehasonlításhoz kiszámítottuk az átlagos négyzetes eltérést:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\text{gateway},i} - \hat{y}_{\text{szenzor},i}\right)^2. \tag{9}$$

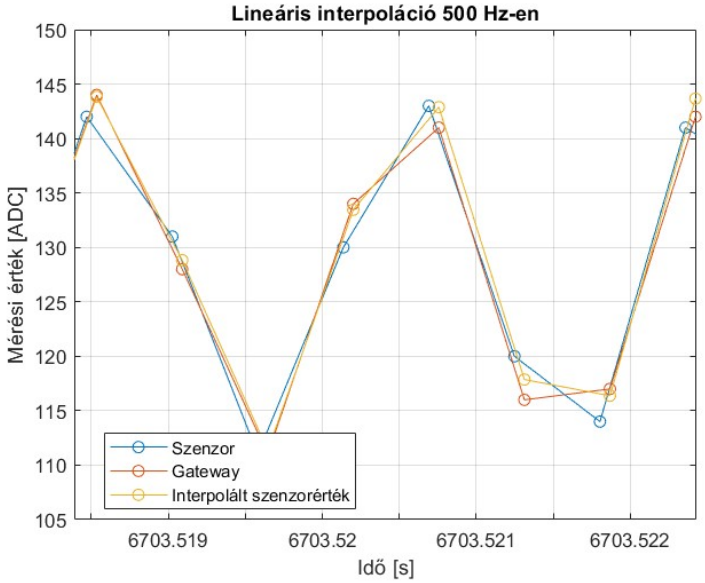
értékeivel hasonlítottuk össze. Az összehasonlításhoz ismét kiszámítottuk az átlagos négyzetes elté-

rést.

Az összehasonlításból kihagytuk azokat a mintákat, amelyekhez nem tartozott érvényes interpolált szenzorérték, valamint a mérés elején és végén található 50-50 mintát. Ezzel elkerültük az időtartomány szélén fellépő interpolációs és szűrési hatások torzító hatását.
Az átlagos négyzetes eltérés értékének 50 [Hz]-es jel esetén 1.34037, 500 Hz-es jel esetén pedig 1.57142-t kaptunk. Ezek alapján megállapítható, hogy 500 [Hz]-es jel esetén az interpoláció eredményei jobbak, mint az 1.5. feladatban bemutatott módszer esetén. Az interpoláció eredményei az alábbi ábrákon láthatóak:



14. ábra. Lineáris interpoláció 50 Hz-en.



15. ábra. Lineáris interpoláció 500 Hz-en.

1.7. feladat

A hetedik feladatban a szenzorértékeket lokális polinomos interpolációval becsültük meg a gateway mintavételi időpontjaiban, minden gateway-mintához a hozzá legközelebb eső szenzorminta környezetében lévő 11 pontját használtuk fel. A feldolgozást az 50 [Hz]-es és az 500 [Hz]-es mérésre is elvégeztük, majd a korábbi feladatokban olvashtó időbélyeg transzformációt is elvégeztük.

Az átszámított gateway-időpontokhoz megkerestük a legközelebbi szenzormintát. Ezt a MATLAB `interp1` függvényével, a `nearest` módszer használatával valósítottuk meg. A legközelebbi szenzorminta indexe jelölte ki a lokális polinom középpontját. A lokális környezet méretét a kódban 5-nek választottuk, így jött ki a középponttal együtt a 11 minta, amelyhez $2 \cdot 5 = 10$-ed fokú polinomot illesztettünk.

Az illesztést nem a nagy abszolút időértékekkel, hanem a gateway keresett időpontjához képesti lokális időeltérésekkel végeztük. A kiszámoltuk szenzormintákhoz tartozó időeltéréseket, majd ezeket a numerikai stabilitás javítása érdekében a legnagyobb abszolút értékkel is normáltuk. Ezután a MATLAB `polyfit` függvényével a normált lokális időkoordinátákra és a szenzorértékekre illesztettük a polinomot:

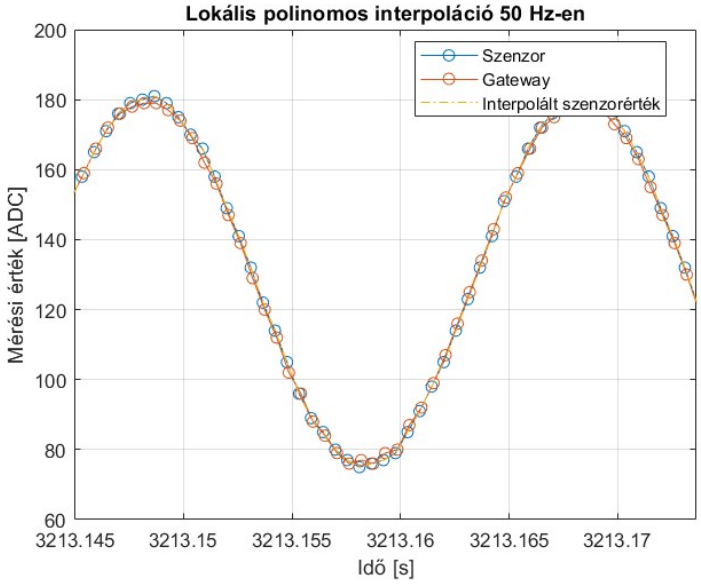
$$y_i \approx a_{10} x_i^{10} + a_9 x_i^9 + \ldots + a_1 x_i + a_0. \tag{10}$$

Mivel a gateway mintavételi időpontja a normált lokális időkoordinátán $x = 0$-nak felel meg, a keresett szenzorértéket az illesztett polinom $x = 0$ helyen felvett értékeként számítottuk ki. Így minden gateway mintához egy, a környező szenzorminták alakját figyelembe vevő lokális becslést kaptunk.
A mérési tartomány elején és végén nem áll rendelkezésre minden szükséges szomszédos pont, ezekhez az értékekhez a program `NaN` értéket rendelt, és az összehasonlítás során ezeket az értékeket figyelmen kívül hagytuk.
Az interpoláció minőségét a gateway mért értékei és a lokális polinommal becsült szenzorértékek közötti átlagos négyzetes eltéréssel jellemeztük:

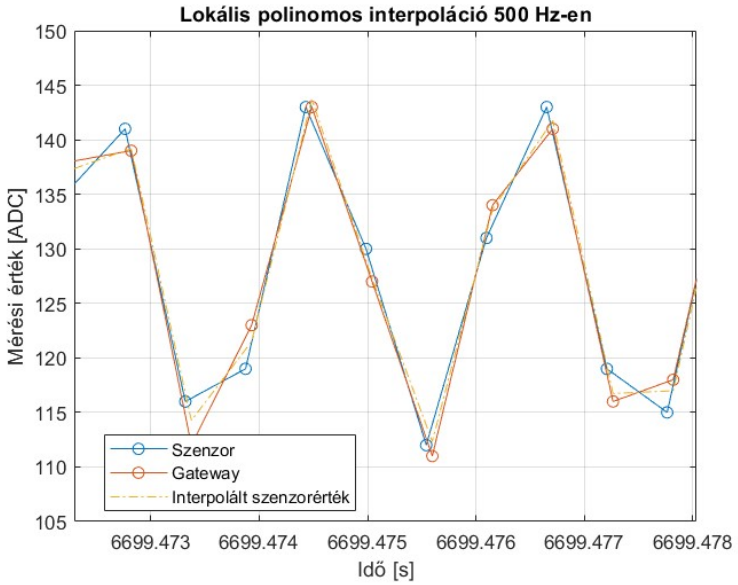
$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} \left(y_{\mathrm{gateway},i} - \hat{y}_{\mathrm{szenzor},i} \right)^2. \tag{11}$$

Az átlagos négyzetes eltérés értékének 50 Hz-es jel esetén $0,831329$-et, 500 Hz-es jel esetén pedig $0,683739$-et kaptunk. Ezek alapján megállapítható, hogy az interpoláció eredményei jobbak, mint a korábbi feladatokban bemutatott módszer esetén, mert a becslült jel lokális alakját több mérési pont alapján határozza meg,viszont a magas polinomfok és a lokális környezet nem megfelelő megválasztása oszcillációt vagy numerikai érzékenységet okozhat. Az interpoláció eredményei az alábbi ábrákon láthatóak: becsült szenzorértékek közötti átlagos négyzetes eltéréssel jellemeztük.
Az átlagos négyzetes eltérés értékének 50 [Hz]-es jel esetén 0.831329-et, 500 [Hz]-es jel esetén pedig 0.683739-et kaptunk. Ezek alapján megállapítható, hogy az interpoláció eredményei jobbak magas frekvencián, mint a korábbi feladatokban bemutatott módszer esetén, mert a becslült jel lokális alak-

ját több mérési pont alapján határozza meg,viszont a magas polinomfok és a lokális környezet nem megfelelő megválasztása oszcillációt vagy numerikai érzékenységet okozhat. Alacsony frekvencián még ilyen magas fokú polinommal sem sikerült jobbat produkálni az 1.5 feladat eredménye. Az interpoláció eredményei az alábbi ábrákon láthatóak:



16. ábra. Interpoláció 10. fokú polinommal 50 Hz-en.



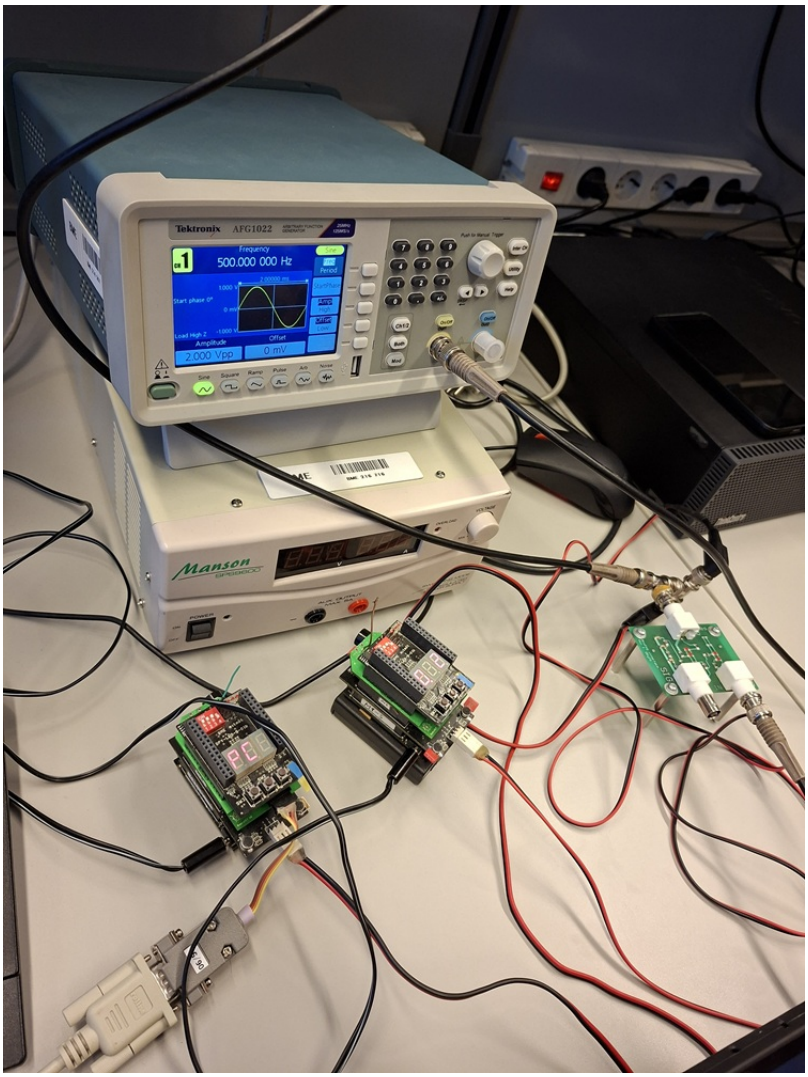
17. ábra. Interpoláció 10. fokú polinommal 500 Hz-en.

2. feladat

A második főfeladatban egy rezonátor alapú tömörítést alkalmazó mérőrendszert vizsgáltunk. A mérőrendszer elrendezése az első főfeladathoz hasonló, viszont ebben a feladatban a szenzor nem a jelet továbbítja, hanem csak annak Fourier-együtthatóit. Ezáltal csökken az átküldendő adatmennyiség, viszont a jel visszaállításához több utólagos számítás szükséges.

2.1. feladat

Az első feladatban összeállítottuk a mérési elrendezést, a szenzor nodeot a 2-es üzemmódban elindítva, hogy a Fourier-együtthatókat továbbítsa a PC felé. Ezt követően a szenzoron és a gatewayen is a megfelelő állapotba kapcsoltuk a DIP kapcsolókat. A mérési elrendezés az alábbi 18. ábra ábrán látható:

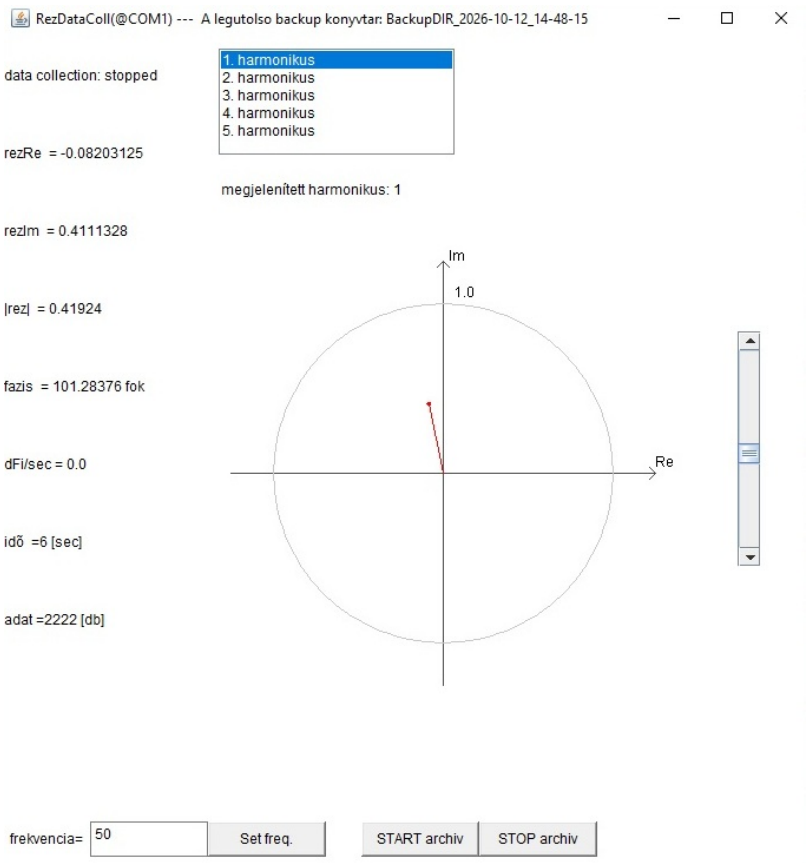


18. ábra. A mérési elrendezés.

Ezt követően a PC-n elindítottuk a mérési programot, majd a függvénygenerátoron beállítottuk a 50 Hz-es szinuszos jelet és kipróbáltuk a szoftver működését.

2.2. feladat

Az 50 Hz-es szinuszos jelet kapcsoltunk először a rendszerre. Az a) feladatban kipróbáltuk, hogy hogyan tudjuk a jel frekvenciáját meghatározni a mérési szoftver segítségével. Egy kis rezonátor frekvenciáról indulva elkezdtük növelni addig amíg az alapharmonikus fázisa konstans értéket nem mutatott. Ez 50 Hz-nél következett be amit a 19. ábra szemléltet.



19. ábra. Konstans alapharmonikus fázis.

Ezt követően a b) feladatban a MATLAB `tic` és `toc` függvényeit használva is meghatároztuk a jel frekvenciáját megközelítőleg, mivel egy ilyen szemre történő mérésnél nehéz pontosan meghatározni a jel frekvenciáját.

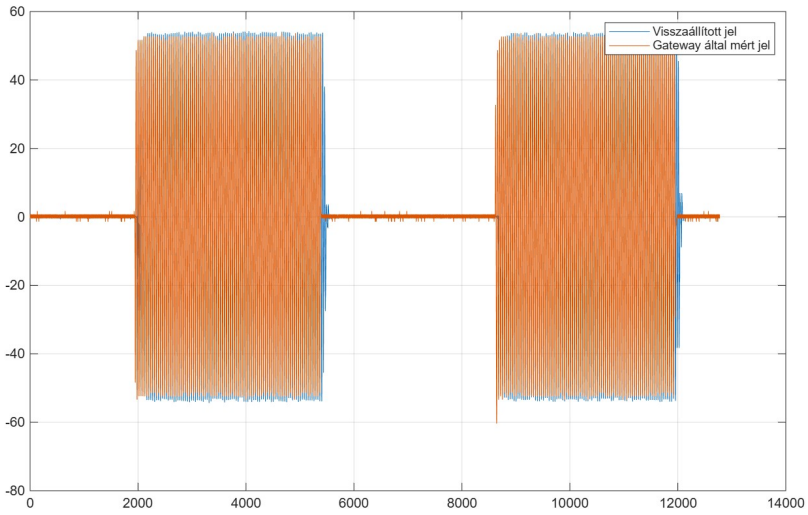
Végül a c) feladatban az `angle` és `unwrap` függvények segítségével meghatároztuk a jel fázisát, majd a `polyfit` függvénnyel lineáris illesztést végeztünk a fázisra. Ezt követően a meredekséget elosztva a 2π-vel megkaptuk a jel frekvenciáját, ami 50 Hz lett. Végül a c) feladatban az `angle` és `unwrap` függvények segítségével meghatároztuk a jel fázisát, majd a `polyfit` függvénnyel lineáris illesztést végeztünk a fázisra. Ezt követően a meredekséget elosztva a 2π-vel megkaptuk a jel frekvenciaeltérését. Ez akkor lett 0 ha 50 [Hz] lett beállítva a szoftverben.

2.3. feladat

A feladat első lépéseként mérési regisztrátumot készítettünk úgy, hogy a jelgenerátort bekapcsoltuk, kikapcsoltuk, bekapcsoltuk, majd megint kikapcsoltuk. A mérés során a jelgenerátor frekvenciáját 50 Hz-en hagytuk, az amplitúdót pedig 2 V-ra állítottuk. A mérési regisztrátumot a `preprocessResonatorFiles.m` script futtatásával dolgoztuk fel.

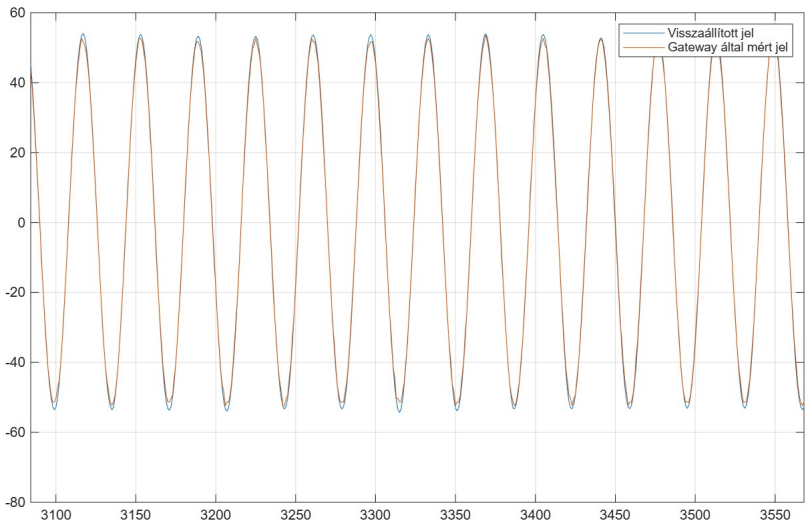
Ezt követően eltávolítottuk a gateway adatainak DC komponensét, majd a korábbi főfeladathoz hasonlóan elvégeztük az időbélyeg transzformációt, tehát átkonvertáltuk a sensor időbélyegeit a gateway időbélyegeire.

Következő feladatrész a jel visszaállítása a Fourier-együtthatókból volt. Mivel a Fourier-együtthatók sokkal ritkábban érkeznek mint a gateway mintavételezési idejei, elsőként megkellett határozni azt, hogy melyik Fourier-csomag van legközelebb az adott kiértékelési időponthoz, hogy a megfelelől Fourier-együtthatókat használhassuk a jel visszaállításához. Ezután következhetett a jelkomponensek fázisainak, majd a bázisfüggvényeknek a meghatározása, majd a jel visszaállítása. A visszaállított jelet és a gateway által mért jelet a(z) 20. ábra szemléltettük:



20. ábra. A visszaállított jel és a gateway által mért jel.

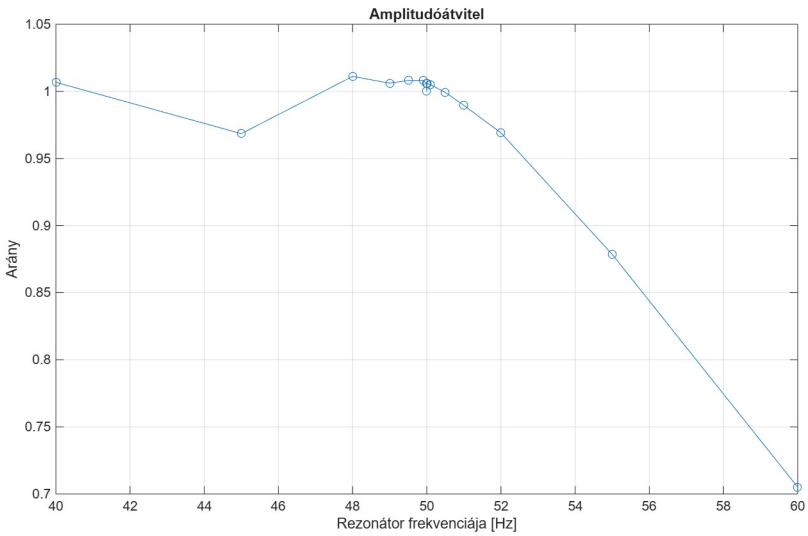
Belenagyítva a visszaállított jel és a gateway által mért jel közötti eltérést a(z) 21. ábra ábrán szemléltettük:



21. ábra. A visszaállított jel és a gateway által mért jel belenagyítva.

2.4. feladat

A feladat első lépéseként a jelgenerátor frekvenciáját 50 Hz-en hagytuk és a rezonátor frekvenciáját fokozatosan elállítottuk mindkét irányba ± 0.01 Hz, ± 0.1 Hz, ± 0.5 Hz, ± 1 Hz, ± 2 Hz, ± 5 Hz és ± 10 Hz-re. A mérés során a jelgenerátor amplitúdóját 2 V-ra állítottuk, majd a mérési regisztrátumot a `preprocessResonatorFiles.m` script futtatásával dolgoztuk fel.
Az 50 Hz-es amplitudóátvitellel normalizálva a(z) 22. ábra-n látható az amplitudóátvitel a rezonátor frekvenciájának függvényében.

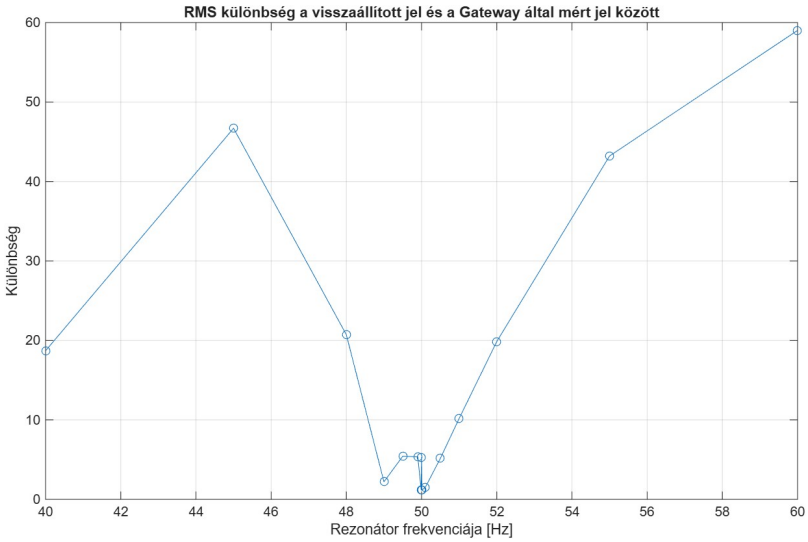


22. ábra. Az amplitudóátvitel a rezonátor frekvenciájának függvényében.

A DFT-vel összehasonlítva a módszert, az eredmény éppen ellentétes jellegű. Amíg a DFT-nél a valós frekvencián a legnagyobb az amplitudó, addig a rezonátor esetében a valós frekvencia közelében volt a legkisebb, közelében elkezdett növekedni, majd távolodva újra csökkenni. A 40 Hz-es mérést valószínleg hiba terhelheti.

2.5. feladat

Ebben a feladatban a korábbi méréseket felhasználva megvizsgáltuk az eltérést a visszaállított jel és a gateway által mért jel között. A frekvencia függvényében alakuló RMS eltéréseket a(z) 23. ábra-n szemléltettük.



23. ábra. Az eltérés a visszaállított jel és a gateway által mért jel között.

Ahogy látható, kis eltérés esetén még elfogadható hibával visszaállítható a jel.

2.6. feladat

A hatodik feladatban a gateway által sűrűn mintavételezett időtartománybeli jelből becsültük a jel Fourier-együtthatóit, majd az ezekből visszaállított jelet összehasonlítottuk a gateway által mért jellel. A feladat célja annak vizsgálata volt, hogy a rezonátoros megfigyelő képes-e a gateway-adatok alapján előállítani a szenzor Fourier együtthatóit.

A szenzor által továbbított első öt harmonikushoz tartozó Fourier-együtthatókat a a valós és képzetes részből állítottuk elő, a korábbi feladatokhoz hasonló módon.A jelek DC-komponensét a feldolgozás elején eltávolítottuk, majd időbélyeg transzformációt végeztünk, szintén a korábbi feladatok során már ismertetett módszer segítségével.

Ezután minden gateway mintához megkerestük a legközelebbi szenzor-adatcsomagot.A MATLAB `interp1` és `nearest` függvényéve. Ebből következik, hogy a szenzor által szolgáltatott együtthatók összehasonlító ábrája lépcsőzetes, mivel egy-egy szenzorcsomag több egymást követő gateway-mintához is hozzárendelésre kerül.

A Fourier-bázis fázisát a szenzor által küldött referenciafázisból számítottuk, a 2.3 feladat során már korábban felhasznált módszer segítségével, majd a meghatározott fázisokból szintén a 2.3 feladatban felhasznált módszerrel a bázisfüggvények is meghatározhatóak voltak. Ezek összefüggései az alábbi 12. és 13. egyenletekben láthatóak:

$$\varphi_1(t) = \varphi_1(t_r) + 2\pi f(t - t_r). \tag{12}$$

Az i-edik harmonikus bázisfüggvénye:

$$c_i(t) = e^{\mathrm{j}i\varphi_1(t)}, \qquad i = 1,\ldots,5. \tag{13}$$

A valós, DC-mentes jel a pozitív frekvenciájú Fourier-együtthatókból az alábbi, már korábban felhasznált összefüggés segítségével állítható elő:

$$y(t) = 2\,\mathrm{Re}\left\{\sum_{i=1}^{5} x_i(t)c_i(t)\right\}. \tag{14}$$

Az együtthatók becsléséhez a mérési útmutatóban megismert rezonátoros megfigyelőt használtuk fel. Először minden gateway-mintára létrehoztuk az aktuális komplex bázisvektort:

$$\boldsymbol{c}_n = \left[e^{\mathrm{j}\varphi_n}, e^{\mathrm{j}2\varphi_n}, \ldots, e^{\mathrm{j}5\varphi_n}\right]^T. \tag{15}$$

A jegyzőkönyv (8)–(10) egyenletei alapján először kiszámítottuk a Fourier-együtthatók által becsült jelet:

$$y'_n = 2\,\mathrm{Re}\left\{\boldsymbol{c}_n^T \hat{\boldsymbol{x}}_n\right\}. \tag{16}$$

Ezután meghatároztuk a gateway-mérés és a becslés különbségét:

$$e_n = y_n - y'_n. \tag{17}$$

A becslési hiba alapján frissítettük a komplex Fourier-együtthatók vektorát:

$$\hat{\boldsymbol{x}}_{n+1} = \hat{\boldsymbol{x}}_n + \alpha e_n \boldsymbol{c}_n^*. \tag{18}$$

A számítások során a bátorsági tényezőt

$$\alpha = 0{,}01 \tag{19}$$

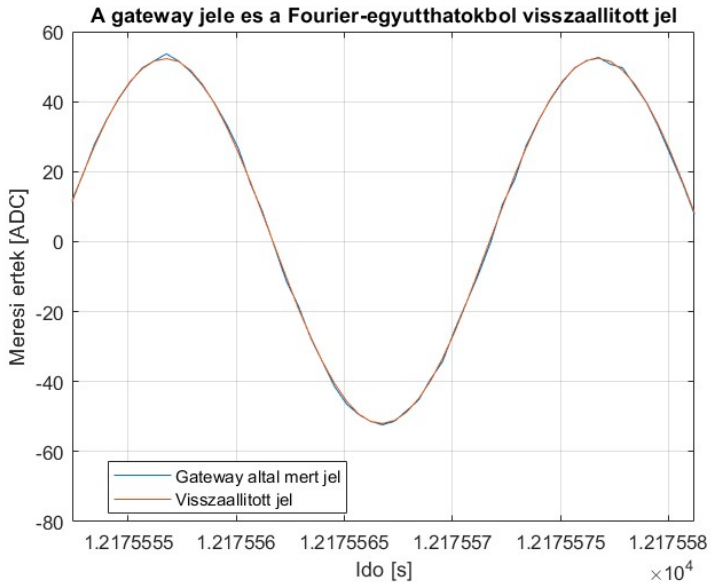
értékre állítottuk. Erre a döntésre kísérleti úton jutottunk, ugyanis ezzel az értékkel kaptuk a legkisebb RMS eltérést. A bátorsági tényező növelése esetén a megfigyelő gyorsabban reagált a jel amplitúdójának változására, viszont a felfutó és lecsengő tranziens szakaszokban túllövés jelent meg.

A Fourier-együtthatók kezdeti becslését nulláról indítottuk, emiatt a jel be- és kikapcsolásakor a rekonstruált jel nem azonnal éri el a gateway-jel amplitúdóját, hanem fokozatosan áll be. Ez a rezonátoros megfigyelő beállási tranziensből származó eltérés.

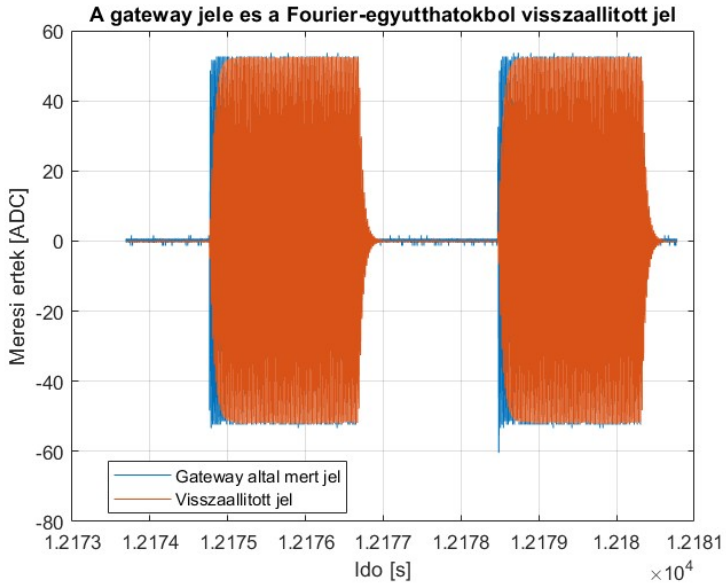
Az algoritmus eredményeit vizuális kettő ábrán vizsgáltuk, a 23. és 24. ábrában a gateway által mért időtartománybeli jelet és az öt Fourier-együtthatóból visszaállított jelet vetettük össze, a 25. ábránpedig a gateway-adatokból becsült, valamint a szenzor által szolgáltatott Fourier-együtthatók

amplitúdóját hasonlítottuk össze.

Az alábbi 24. ábrán látható a fentebb említett eltérés a felfutási és lecsengési tranziens esetén, viszont a 23. ábrán nagyítás után az állandósult szakaszon csak minimális eltérés látható a két jel között.

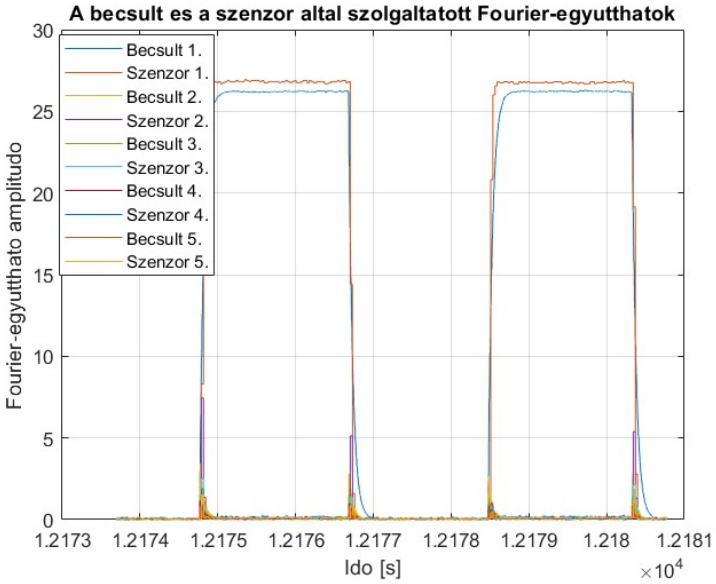


24. ábra. Gateway jel és visszaállított jel belenagyítva.



25. ábra. Gateway jel és visszaállított jel.

A 25. ábrán a gateway-adatokból becsült, valamint a szenzor által szolgáltatott Fourier-együtthatók amplitúdójának összehasonlítása látható. Az első harmonikus amplitúdója volt a legnagyobb, ezért annak abszolút hibája is nagyobb lett, illetve szintén a fentebb leírt tranzienseknél jelentkező különbség is látható.



26. ábra. Becsült és szenzor által szolgáltatott Fourier-együtthatók amplitúdója.

Az összehasonlítást a komplex együtthatók és a gateway-jel és a visszaállított jel közötti eltérést RMS értékével számítottuk.

A mérés során a gateway-jel és a visszaállított jel RMS eltérése 4,74589 volt. Az egyes harmonikusokra kapott RMS eltérések:

Harmonikus	RMS eltérés
1.	1,51301
2.	0,631144
3.	0,255675
4.	0,1732
5.	0,160554

(20)

Az eredmények alapján megállapítható, hogy a rezonátoros megfigyelő az állandósult szakaszban megfelelően közelíti a gateway által mért jelet. Az eltérés döntő része a jel ki- és bekapcsolásakor jelentkező tranziens szakaszokból, a becslő beállási idejéből, a mérési zajból, valamint a szenzor és a gateway eltérő mintavételi időpontjaiból származhat.

Függelék

MATLAB kódok

Az 1.1-4 mérés MATLAB-kódja

```
%% Clearing wokspace
clear;
clf;
clc;
close all;

%% Loading measurement files and parameters for 50 [Hz] measurement
gwySampFull = load("Measurements/ElsoFeladat50Hz/pre_gwySampFull.dat");
sensorDataFull = load("Measurements/ElsoFeladat50Hz/pre_sensorDataFull.
   dat");
stmpSync = load("Measurements/ElsoFeladat50Hz/pre_stmpSync.dat");

sensorDataX = sensorDataFull(:, 1);
sensorDataY = sensorDataFull(:, 2);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Plotting the raw data
figure;
plot(sensorDataX, sensorDataY);
hold on;
plot(gwySampX, gwySampY);
xlabel("Idő [s]");
ylabel("Mérési érték [ADC]");
title("Nyers szenzor- és gateway-adatok (50 Hz)");
legend("Szenzor", "Gateway", "Location", "best");
grid on;

figure;
plot(stmpSyncGwy, stmpSyncSens);
xlabel("Gateway időbélyege [s]");
```

```
38 ylabel("Szenzor időbélyege [s]");
39 title("Nyers szinkronizációs időbélyegek (50 Hz)");
40 grid on;
41
42 %% Időbélyeg-transzformáció
43 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
44 GwyToSensorTime  = polyval(idobelyegPoli, gwySampX);
45
46 %% Plot offset data
47 figure;
48 plot(sensorDataX, sensorDataY);
49 hold on;
50 plot(GwyToSensorTime , gwySampY);
51 xlabel("Idő [s]");
52 ylabel("Mérési érték [ADC]");
53 title("Időbélyeggel összehangolt adatok (50 Hz)");
54 legend("Szenzor", "Gateway", "Location", "best");
55 grid on;
56
57 %% Clearing workspace
58 clearvars;
59
60 %% Loading measurement files and parameters for 500 [Hz] measurement
61 gwySampFull = load("Measurements/ElsoFeladat500Hz/pre_gwySampFull.dat")
      ;
62 sensorDataFull = load("Measurements/ElsoFeladat500Hz/pre_sensorDataFull
      .dat");
63 stmpSync = load("Measurements/ElsoFeladat500Hz/pre_stmpSync.dat");
64
65 sensorDataX = sensorDataFull(:, 1);
66 sensorDataY = sensorDataFull(:, 2);
67 clearvars sensorDataFull;
68
69 gwySampX = gwySampFull(:, 1);
70 gwySampY = gwySampFull(:, 2);
71 clearvars gwySampFull;
72
73 stmpSyncGwy = stmpSync(:, 1);
74 stmpSyncSens = stmpSync(:, 2);
75 clearvars stmpSync;
76
77 %% Plotting the raw data
78 figure;
```

```
79 plot(sensorDataX, sensorDataY);
80 hold on;
81 plot(gwySampX, gwySampY);
82 xlabel("Idő [s]");
83 ylabel("Mérési érték [ADC]");
84 title("Nyers szenzor- és gateway-adatok (500 Hz)");
85 legend("Szenzor", "Gateway", "Location", "best");
86 grid on;
87
88 figure;
89 plot(stmpSyncGwy, stmpSyncSens);
90 xlabel("Gateway időbélyege [s]");
91 ylabel("Szenzor időbélyege [s]");
92 title("Nyers szinkronizációs időbélyegek (500 Hz)");
93 grid on;
94
95 %% Időbélyeg-transzformáció
96 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
97 GwyToSensorTime  = polyval(idobelyegPoli, gwySampX);
98
99 %% Plot offset data
100 figure;
101 plot(sensorDataX, sensorDataY);
102 hold on;
103 plot(GwyToSensorTime , gwySampY);
104 xlabel("Idő [s]");
105 ylabel("Mérési érték [ADC]");
106 title("Időbélyeggel összehangolt adatok (500 Hz)");
107 legend("Szenzor", "Gateway", "Location", "best");
108 grid on;
109
110 %% Clearing workspace
111 clearvars;
112
113 %% Loading measurement files and parameters for 50 [Hz] measurement
       with changing amplitudes
114 gwySampFull = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_gwySampFull.dat");
115 sensorDataFull = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_sensorDataFull.dat");
116 stmpSync = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_stmpSync.dat");
117
```

```
118 sensorDataX = sensorDataFull(:, 1);
119 sensorDataY = sensorDataFull(:, 2);
120 clearvars sensorDataFull;
121
122 gwySampX = gwySampFull(:, 1);
123 gwySampY = gwySampFull(:, 2);
124 clearvars gwySampFull;
125
126 stmpSyncGwy = stmpSync(:, 1);
127 stmpSyncSens = stmpSync(:, 2);
128 clearvars stmpSync;
129
130 %% Plotting the raw data
131 figure;
132 plot(sensorDataX, sensorDataY);
133 hold on;
134 plot(gwySampX, gwySampY);
135 xlabel("Idő [s]");
136 ylabel("Mérési érték [ADC]");
137 title("Nyers szenzor- és gateway-adatok (50 Hz, változó amplitúdó)");
138 legend("Szenzor", "Gateway", "Location", "best");
139 grid on;
140
141 figure;
142 plot(stmpSyncGwy, stmpSyncSens);
143 xlabel("Gateway időbélyege [s]");
144 ylabel("Szenzor időbélyege [s]");
145 title("Nyers szinkronizációs időbélyegek (50 Hz, változó amplitúdó)");
146 grid on;
147
148 %% Időbélyeg-transzformáció
149 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
150 GwyToSensorTime = polyval(idobelyegPoli, gwySampX);
151
152 %% Plot offset data
153 figure;
154 plot(sensorDataX, sensorDataY);
155 hold on;
156 plot(GwyToSensorTime , gwySampY);
157 xlabel("Idő [s]");
158 ylabel("Mérési érték [ADC]");
159 title("Időbélyeggel összehangolt adatok (50 Hz, változó amplitúdó)");
160 legend("Szenzor", "Gateway", "Location", "best");
```

```
161 grid on;
162
163 %% Pseudoinvers calc
164
165 X = [stmpSyncGwy, ones(size(stmpSyncGwy))];
166 Y = stmpSyncSens;
167 cond(X)
168 idobelyegPoliPseudo = pinv(X) * Y;
169 GwyToSensorTime_pseudo = polyval(idobelyegPoliPseudo, gwySampX);
170
171 %% Plot offset data
172 figure;
173 plot(sensorDataX, sensorDataY);
174 hold on;
175 plot(GwyToSensorTime_pseudo, gwySampY);
176 xlabel("Idő [s]");
177 ylabel("Mérési érték [ADC]");
178 title("Összehangolt adatok pszeudoinverzzel (50 Hz, változó amplitúdó)
    ");
179 legend("Szenzor", "Gateway", "Location", "best");
180 grid on;
181
182 %% Szinkronizációs pont eltolás
183 stmpSyncGwy = stmpSyncGwy + 10^6;
184 gwySampX_shifted = gwySampX + 10^6;
185
186 %% Pseudoinvers calc
187
188 Y = stmpSyncSens;
189 X_shifted = [stmpSyncGwy, ones(size(stmpSyncGwy))];
190 cond(X_shifted)
191
192 timeMean = mean(stmpSyncGwy);
193 X_centered = [stmpSyncGwy - timeMean, ones(size(stmpSyncGwy))];
194 cond(X_centered)
195 idobelyegPoliPseudo = pinv(X_centered) * Y;
196 GwyToSensorTime_pseudo = polyval(idobelyegPoliPseudo, gwySampX_shifted
    - timeMean);
197
198 %% Plot offset data
199 figure;
200 plot(sensorDataX, sensorDataY);
201 hold on;
```

```
plot(GwyToSensorTime_pseudo, gwySampY);
xlabel("Idő [s]");
ylabel("Mérési érték [ADC]");
title("Pszeudoinverzes összehangolás 10^6 s időeltolással");
legend("Szenzor", "Gateway", "Location", "best");
grid on;
```

Az 1.5 mérés MATLAB-kódja

```
%% Az 1.5 feladat: lineáris interpoláció
clear;
clf;
clc;
close all;

frequencies = [50, 500];
measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];

for k = 1:numel(frequencies)
    frequency = frequencies(k);
    measurementFolder = measurementFolders(k);

    gwySampFull = load(fullfile("Measurements", measurementFolder, ...
        "pre_gwySampFull.dat"));
    sensorDataFull = load(fullfile("Measurements", measurementFolder,
       ...
        "pre_sensorDataFull.dat"));
    stmpSync = load(fullfile("Measurements", measurementFolder, ...
        "pre_stmpSync.dat"));

    sensorDataX = sensorDataFull(:, 1);
    sensorDataY = sensorDataFull(:, 2);
    gwySampX = gwySampFull(:, 1);
    gwySampY = gwySampFull(:, 2);

    idobelyegPoli = polyfit(stmpSync(:, 1), stmpSync(:, 2), 1);
    gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);


    sensorDataYInterpolated = interp1(sensorDataX, sensorDataY,
       gatewayTimeInSensorClock, "linear");

    validSamples = ~isnan(sensorDataYInterpolated);
    squaredError = mean((gwySampY(validSamples)  -
```

```
        sensorDataYInterpolated(validSamples)).^2);
    fprintf("%d Hz: átlagos négyzetes eltérés = %.6g\n", frequency,
        squaredError);

    figure;
    plot(sensorDataX, sensorDataY, "-o");
    hold on;
    plot(gatewayTimeInSensorClock, gwySampY, "-o");
    plot(gatewayTimeInSensorClock, sensorDataYInterpolated, "-.");
    xlabel("Idő [s]");
    ylabel("Mérési érték [ADC]");
    title(sprintf("Lineáris interpoláció %d Hz-en", frequency));
    legend("Szenzor", "Gateway", ...
        "Interpolált szenzorérték", "Location", "best");
    grid on;
end
```

Az 1.6 mérés MATLAB-kódja

```
%% Az 1.6 feladat: felmintavételezés
%% Clearing wokspace
clear;
clf;
clc;
close all;


frequencies = [50, 500];
measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];

for k = 1:numel(frequencies)
    frequency = frequencies(k);
    measurementFolder = measurementFolders(k);

    %% Loading measurement files and parameters
    gwySampFull = load(fullfile("Measurements", measurementFolder, "
        pre_gwySampFull.dat"));
    sensorDataFull = load(fullfile("Measurements", measurementFolder, "
        pre_sensorDataFull.dat"));
    stmpSync = load(fullfile("Measurements", measurementFolder, "
        pre_stmpSync.dat"));

    sensorDataX = sensorDataFull(:, 1);
    sensorDataY = sensorDataFull(:, 2);
```

```
    gwySampX = gwySampFull(:, 1);
    gwySampY = gwySampFull(:, 2);

    stmpSyncGwy = stmpSync(:, 1);
    stmpSyncSens = stmpSync(:, 2);

    %% Upsampling the sensor data

    upSampNum = 10;
    Ts = mean(diff(sensorDataX));
    fs = 1 / Ts;
    fsNew = fs * upSampNum;

    yUp = zeros(upSampNum * (numel(sensorDataY) - 1) + 1, 1);
    yUp(1:upSampNum:end) = sensorDataY;

    cutoffFrequency = fs / 2;
    [b, a] = butter(4, cutoffFrequency / (fsNew / 2), "low");

    upSampledSensorDataY = filtfilt(b, a, yUp);

    upSampledSensorDataY = upSampNum * upSampledSensorDataY;

    upSampledSensorDataX = sensorDataX(1) + (0:numel(yUp) - 1)' / fsNew
        ;

    %% Interpolation
    idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
    gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);
    sensorDataYInterpolated = interp1(upSampledSensorDataX,
        upSampledSensorDataY, gatewayTimeInSensorClock, 'linear');

    validSamples = ~isnan(sensorDataYInterpolated);

    edgeSamples = 50;
    comparisonSamples = validSamples;
    comparisonSamples(1:edgeSamples) = false;
    comparisonSamples(end-edgeSamples + 1:end) = false;

    squaredError = mean((gwySampY(comparisonSamples) - ...
        sensorDataYInterpolated(comparisonSamples)).^2);
    fprintf("%d Hz: átlagos négyzetes eltérés = %.6g\n", frequency,
        squaredError);
```

```
63
64      %% Plotting
65      figure;
66      plot(sensorDataX, sensorDataY, "-o");
67      hold on;
68      plot(gatewayTimeInSensorClock(comparisonSamples), ...
69          gwySampY(comparisonSamples), "-o");
70      plot(gatewayTimeInSensorClock(comparisonSamples), ...
71          sensorDataYInterpolated(comparisonSamples), "-o");
72      xlabel("Idő [s]");
73      ylabel("Mérési érték [ADC]");
74      title(sprintf("Lineáris interpoláció %d Hz-en", frequency));
75      legend("Szenzor", "Gateway", ...
76          "Interpolált szenzorérték", "Location", "best");
77      grid on;
78  end
```

Az 1.7 mérés MATLAB-kódja

```
1   %% Az 1.7 feladat: lokális polinomos interpoláció
2   clear;
3   clf;
4   clc;
5   close all;
6
7   frequencies = [50, 500];
8   measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];
9   L = 5;
10  polynomialDegree = 2 * L;
11
12  for k = 1:numel(frequencies)
13      frequency = frequencies(k);
14      measurementFolder = measurementFolders(k);
15
16      gwySampFull = load(fullfile("Measurements", measurementFolder, "
            pre_gwySampFull.dat"));
17      sensorDataFull = load(fullfile("Measurements", measurementFolder, "
            pre_sensorDataFull.dat"));
18      stmpSync = load(fullfile("Measurements", measurementFolder, "
            pre_stmpSync.dat"));
19
20      sensorDataX = sensorDataFull(:, 1);
21      sensorDataY = sensorDataFull(:, 2);
22      gwySampX = gwySampFull(:, 1);
```

```
23      gwySampY = gwySampFull(:, 2);
24
25      idobelyegPoli = polyfit(stmpSync(:, 1), stmpSync(:, 2), 1);
26      gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);
27
28
29      nearestSensorIndex = nan(size(gatewayTimeInSensorClock));
30      inSensorRange = gatewayTimeInSensorClock >= sensorDataX(1) &
           gatewayTimeInSensorClock <= sensorDataX(end);
31      nearestSensorIndex(inSensorRange) = interp1(sensorDataX, (1:numel(
           sensorDataX))', gatewayTimeInSensorClock(inSensorRange), "
           nearest");
32
33      sensorDataYInterpolated = nan(size(gatewayTimeInSensorClock));
34      validSamples = false(size(gatewayTimeInSensorClock));
35      for sampleIndex = find(inSensorRange)'
36          centerIndex = nearestSensorIndex(sampleIndex);
37          if centerIndex - L < 1 || centerIndex + L > numel(sensorDataX)
38              continue;
39          end
40
41          localIndices = (centerIndex - L):(centerIndex + L);
42          localTimeOffsets = sensorDataX(localIndices)  -
               gatewayTimeInSensorClock(sampleIndex);
43          localTimeScale = max(abs(localTimeOffsets));
44          localPolynomial = polyfit(localTimeOffsets / localTimeScale,
               sensorDataY(localIndices), polynomialDegree);
45          sensorDataYInterpolated(sampleIndex) = polyval(localPolynomial,
                0);
46          validSamples(sampleIndex) = true;
47      end
48
49      squaredError = mean((gwySampY(validSamples) -
           sensorDataYInterpolated(validSamples)).^2);
50      fprintf("%d Hz (L = %d, %d. fokú polinom): átlagos négyzetes eltéré
           s = %.6g\n", frequency, L, polynomialDegree, squaredError);
51
52      figure;
53      plot(sensorDataX, sensorDataY, "-o");
54      hold on;
55      plot(gatewayTimeInSensorClock(validSamples), gwySampY(validSamples)
           , "-o");
56      plot(gatewayTimeInSensorClock(validSamples),
```

```
        sensorDataYInterpolated(validSamples), "-.");
    xlabel("Idő [s]");
    ylabel("Mérési érték [ADC]");
    title(sprintf("Lokális polinomos interpoláció %d Hz-en", frequency)
        );
    legend("Szenzor", "Gateway", ...
        "Interpolált szenzorérték", "Location", "best");
    grid on;
end
```

Az 2.2 mérés MATLAB-kódja

```
%% Clearing wokspace
clear;
clf;
clc;
close all;

%% Loading measurement files and parameters for 50 [Hz] measurement
gwySampFull = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
    pre_gwySampFull.dat");
sensorDataFull = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
    pre_sensorResonator.dat");
stmpSync = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
    pre_stmpSync.dat");
signalFreq = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
    signalFreq.dat");

sensorDataID = sensorDataFull(:, 1);
sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:,
    9);
sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull
    (:, 10);
sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:,
    11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull
    (:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:,
    13);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
```

```
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% c, feladat
Phase = angle(sensorDataFirstHarmonic);
Phase = unwrap(Phase);
phasePoly = polyfit(sensorDataTime, Phase, 1);
gradient = phasePoly(1) / (2* pi);
```

Az 2.3 mérés MATLAB-kódja

```
%% Clearing wokspace
clear;
clf;
clc;
close all;

%% Loading measurement files and parameters for 50 [Hz] measurement
gwySampFull = load("Measurements/MasodikFeladatJel50HzProgi60Hz/
   pre_gwySampFull.dat");
sensorDataFull = load("Measurements/MasodikFeladatJel50HzProgi60Hz/
   pre_sensorResonator.dat");
stmpSync = load("Measurements/MasodikFeladatJel50HzProgi60Hz/
   pre_stmpSync.dat");
signalFreq = load("Measurements/MasodikFeladatJel50HzProgi60Hz/
   signalFreq.dat");

sensorDataID = sensorDataFull(:, 1);
sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:,
    9);
sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull
   (:, 10);
sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:,
    11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull
   (:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:,
    13);
```

```
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Gateway átlag kivonás
gwySampY = gwySampY - mean(gwySampY);

%% Időbélyeg transzformáció
idobelyegPoli = polyfit(stmpSyncSens, stmpSyncGwy, 1);
sensorToGwyTime  = polyval(idobelyegPoli, sensorDataTime);

evalTimes = zeros(length(gwySampX), 1);
evalValues = zeros(length(gwySampX), 1);
evalDeltaT = zeros(length(gwySampX), 1);

for i = 1:length(gwySampX)
    minVal = 100000;
    minDeltaT = 0;
    minID = 0;
    for j = 1:length(sensorToGwyTime)
        if (abs(gwySampX(i) - sensorToGwyTime(j)) < minVal)
            minID = sensorToGwyTime(j);
            minVal = abs(gwySampX(i) - sensorToGwyTime(j));
            minDeltaT = gwySampX(i) - sensorToGwyTime(j);
        end
    end
    evalTimes(i) = minID;
    evalValues(i) = minVal;
    evalDeltaT(i) = minDeltaT;
end

reconstructedPhasesBase = zeros(length(gwySampX), 1);
reconstructedPhasesFirst = zeros(length(gwySampX), 1);
reconstructedPhasesSecond = zeros(length(gwySampX), 1);
reconstructedPhasesThird = zeros(length(gwySampX), 1);
reconstructedPhasesFourth = zeros(length(gwySampX), 1);
reconstructedPhasesFifth = zeros(length(gwySampX), 1);
```

```
64
65 reconstructedCoefFirst = zeros(length(gwySampX), 1);
66 reconstructedCoefSecond = zeros(length(gwySampX), 1);
67 reconstructedCoefThird = zeros(length(gwySampX), 1);
68 reconstructedCoefFourth = zeros(length(gwySampX), 1);
69 reconstructedCoefFifth = zeros(length(gwySampX), 1);
70
71 for i = 1 : length(reconstructedPhasesBase)
72     for j = 1:length(sensorDataPhi)
73         if sensorToGwyTime(j) == evalTimes(i)
74             reconstructedPhasesBase(i) = sensorDataPhi(j) + evalDeltaT(
                i) * signalFreq * 2 * pi;
75             reconstructedPhasesFirst(i) = sensorDataPhi(j) + evalDeltaT
                (i) * signalFreq * 1 * 2 * pi;
76             reconstructedPhasesSecond(i) = 2 *sensorDataPhi(j) +
                evalDeltaT(i) * signalFreq * 2 * 2 * pi;
77             reconstructedPhasesThird(i) =  3* sensorDataPhi(j) +
                evalDeltaT(i) * signalFreq * 3 * 2 * pi;
78             reconstructedPhasesFourth(i) = 4 * sensorDataPhi(j) +
                evalDeltaT(i) * signalFreq * 4 * 2 * pi;
79             reconstructedPhasesFifth(i) = 5 * sensorDataPhi(j) +
                evalDeltaT(i) * signalFreq * 5 * 2 * pi;
80
81             reconstructedCoefFirst(i) = sensorDataFirstHarmonic(j);
82             reconstructedCoefSecond(i) = sensorDataSecondHarmonic(j);
83             reconstructedCoefThird(i) = sensorDataThirdHarmonic(j);
84             reconstructedCoefFourth(i) = sensorDataFourthHarmonic(j);
85             reconstructedCoefFifth(i) = sensorDataFifthHarmonic(j);
86         end
87     end
88 end
89
90
91 reconstructedBaseFncFirst = abs(reconstructedCoefFirst) .* exp(1j * (
    reconstructedPhasesFirst + angle(reconstructedCoefFirst)));
92 reconstructedBaseFncSecond = abs(reconstructedCoefSecond) .* exp(1j * (
    reconstructedPhasesSecond + angle(reconstructedCoefSecond)));
93 reconstructedBaseFncThird = abs(reconstructedCoefThird) .* exp(1j * (
    reconstructedPhasesThird + angle(reconstructedCoefThird)));
94 reconstructedBaseFncFourth = abs(reconstructedCoefFourth) .* exp(1j * (
    reconstructedPhasesFourth + angle(reconstructedCoefFourth)));
95 reconstructedBaseFncFifth = abs(reconstructedCoefFifth) .* exp(1j * (
    reconstructedPhasesFifth + angle(reconstructedCoefFifth)));
```

```
96
97  reconstructedSignal = 2 * real(reconstructedBaseFncFirst +
        reconstructedBaseFncSecond + ...
98  reconstructedBaseFncThird + reconstructedBaseFncFourth +
        reconstructedBaseFncFifth);
99
100 plot(reconstructedSignal)
101 hold on;
102 plot(gwySampY);
103 legend('Visszaállított␣jel', 'Gateway␣által␣mért␣jel');
104 grid on;
```

Az 2.6 mérés MATLAB-kódja

```
1   <<<<<<< HEAD
2   %% A 2.6 feladat: Fourier-egyutthatok becslese a gateway adataibol
3   =======
4   %% Clearing workspace
5   >>>>>>> 1ef44c8cf8cf978c4bcb4804ed3445458912562c
6   clear;
7   clf;
8   clc;
9   close all;
10
11  <<<<<<< HEAD
12  %% Meresi adatok es parameterek betoltese
13  gwySampFull = load("Measurements/
        MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_gwySampFull.dat");
14  sensorDataFull = load("Measurements/
        MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_sensorResonator.dat")
        ;
15  stmpSync = load("Measurements/
        MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_stmpSync.dat");
16  signalFreq = load("Measurements/
        MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/signalFreq.dat");
17
18  sensorDataID = sensorDataFull(:, 1);
19  sensorDataTime = sensorDataFull(:, 2);
20  sensorDataPhi = sensorDataFull(:, 3);
21  sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:,
        9);
22  sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull
        (:, 10);
23  sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:,
```

```
    11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull
    (:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:,
    13);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Gateway DC komponensenek eltavolitasa
gwySampY = gwySampY - mean(gwySampY);

%% Idobelyeg-transzformacio
idobelyegPoli = polyfit(stmpSyncSens, stmpSyncGwy, 1);
sensorToGwyTime = polyval(idobelyegPoli, sensorDataTime);

%% A szenzor fazisanak meghatarozasa a gateway mintavetelezesi
    idopontjaiban
% A gateway es a szenzor idoskala a szinkronizacios pontok alapjan
    azonos.
inSensorRange = gwySampX >= sensorToGwyTime(1) & gwySampX <=
    sensorToGwyTime(end);
validSamples = false(size(gwySampX));
validSamples(inSensorRange) = true;

nearestSensorIndex = nan(size(gwySampX));
nearestSensorIndex(inSensorRange) = interp1(sensorToGwyTime, (1:numel(
    sensorToGwyTime))', gwySampX(inSensorRange), "nearest");

gatewayPhase = nan(size(gwySampX));
for sampleIndex = find(inSensorRange)'
    sensorIndex = nearestSensorIndex(sampleIndex);
    gatewayPhase(sampleIndex) = sensorDataPhi(sensorIndex) + (gwySampX(
        sampleIndex) - sensorToGwyTime(sensorIndex)) * signalFreq * 2 *
        pi;
end

```

```
%% Fourier-egyutthatok becslese a rezonaros megfigyelovel
% A becsles a jegyzokonyv (8)-(10) egyenletei szerint tortenik:
% y'_n = 2 * real(c_n^T * xHat_n)
% e_n = y_n - y'_n
% xHat_(n+1) = xHat_n + alpha * e_n * conj(c_n)
harmonicNumbers = (1:5)';
alpha = 0.01;

reconstructedCoefFirst = nan(size(gwySampX));
reconstructedCoefSecond = nan(size(gwySampX));
reconstructedCoefThird = nan(size(gwySampX));
reconstructedCoefFourth = nan(size(gwySampX));
reconstructedCoefFifth = nan(size(gwySampX));
reconstructedSignal = nan(size(gwySampX));

estimatedCoefficients = zeros(numel(harmonicNumbers), 1);
validIndices = find(inSensorRange);
for sampleIndex = validIndices'
    currentBasis = exp(1i * harmonicNumbers * gatewayPhase(sampleIndex)
       );
    complexEstimate = currentBasis.' * estimatedCoefficients;
    currentEstimate = 2 * real(complexEstimate);
    currentError = gwySampY(sampleIndex) - currentEstimate;

    reconstructedSignal(sampleIndex) = currentEstimate;
    reconstructedCoefFirst(sampleIndex) = estimatedCoefficients(1);
    reconstructedCoefSecond(sampleIndex) = estimatedCoefficients(2);
    reconstructedCoefThird(sampleIndex) = estimatedCoefficients(3);
    reconstructedCoefFourth(sampleIndex) = estimatedCoefficients(4);
    reconstructedCoefFifth(sampleIndex) = estimatedCoefficients(5);

    estimatedCoefficients = estimatedCoefficients ...
        + alpha * currentError * conj(currentBasis);
end

%% A szenzor egyutthatoinak gateway idopontjaira illesztese
sensorCoefFirstAtGateway = nan(size(gwySampX));
sensorCoefSecondAtGateway = nan(size(gwySampX));
sensorCoefThirdAtGateway = nan(size(gwySampX));
sensorCoefFourthAtGateway = nan(size(gwySampX));
sensorCoefFifthAtGateway = nan(size(gwySampX));

for sampleIndex = find(inSensorRange)'
```

```
100     sensorIndex = nearestSensorIndex(sampleIndex);
101     sensorCoefFirstAtGateway(sampleIndex) = sensorDataFirstHarmonic(
            sensorIndex);
102     sensorCoefSecondAtGateway(sampleIndex) = sensorDataSecondHarmonic(
            sensorIndex);
103     sensorCoefThirdAtGateway(sampleIndex) = sensorDataThirdHarmonic(
            sensorIndex);
104     sensorCoefFourthAtGateway(sampleIndex) = sensorDataFourthHarmonic(
            sensorIndex);
105     sensorCoefFifthAtGateway(sampleIndex) = sensorDataFifthHarmonic(
            sensorIndex);
106 end
107
108 %% Hibak kiszamolasa
109 coefficientDifference = [
110     reconstructedCoefFirst(validSamples) - sensorCoefFirstAtGateway(
            validSamples), ...
111     reconstructedCoefSecond(validSamples) - sensorCoefSecondAtGateway(
            validSamples), ...
112     reconstructedCoefThird(validSamples) - sensorCoefThirdAtGateway(
            validSamples), ...
113     reconstructedCoefFourth(validSamples) - sensorCoefFourthAtGateway(
            validSamples), ...
114     reconstructedCoefFifth(validSamples) - sensorCoefFifthAtGateway(
            validSamples)];
115
116 coefficientRmsError = sqrt(mean(abs(coefficientDifference).^2, 1));
117 signalRmsError = sqrt(mean( ...
118     (gwySampY(validSamples) - reconstructedSignal(validSamples)).^2));
119
120 fprintf("Alfa = %.4f\n", alpha);
121 fprintf("Gateway es a visszaallitott jel RMS elterese = %.6g\n",
        signalRmsError);
122 fprintf("Fourier-egyutthatok RMS elterese:\n");
123 fprintf("  1. harmonikus: %.6g\n", coefficientRmsError(1));
124 fprintf("  2. harmonikus: %.6g\n", coefficientRmsError(2));
125 fprintf("  3. harmonikus: %.6g\n", coefficientRmsError(3));
126 fprintf("  4. harmonikus: %.6g\n", coefficientRmsError(4));
127 fprintf("  5. harmonikus: %.6g\n", coefficientRmsError(5));
128
129 %% A jel visszaallitasa es a gateway mereseinek osszehasonlitasa
130 figure;
131 plot(gwySampX(validSamples), gwySampY(validSamples));
```

```
132 hold on;
133 plot(gwySampX(validSamples), reconstructedSignal(validSamples));
134 xlabel("Ido [s]");
135 ylabel("Meresi ertek [ADC]");
136 title("A gateway jele es a Fourier-egyutthatokbol visszaallitott jel");
137 legend("Gateway altal mert jel", "Visszaallitott jel", ...
138     "Location", "best");
139 grid on;
140
141 %% A becsult es a szenzor altal szolgaltatott egyutthatok
       osszehasonlitasa
142 figure;
143 plot(gwySampX(validSamples), abs(reconstructedCoefFirst(validSamples)))
       ;
144 hold on;
145 plot(gwySampX(validSamples), abs(sensorCoefFirstAtGateway(validSamples)
       ));
146 plot(gwySampX(validSamples), abs(reconstructedCoefSecond(validSamples))
       );
147 plot(gwySampX(validSamples), abs(sensorCoefSecondAtGateway(validSamples
       )));
148 plot(gwySampX(validSamples), abs(reconstructedCoefThird(validSamples)))
       ;
149 plot(gwySampX(validSamples), abs(sensorCoefThirdAtGateway(validSamples)
       ));
150 plot(gwySampX(validSamples), abs(reconstructedCoefFourth(validSamples))
       );
151 plot(gwySampX(validSamples), abs(sensorCoefFourthAtGateway(validSamples
       )));
152 plot(gwySampX(validSamples), abs(reconstructedCoefFifth(validSamples)))
       ;
153 plot(gwySampX(validSamples), abs(sensorCoefFifthAtGateway(validSamples)
       ));
154 xlabel("Ido [s]");
155 ylabel("Fourier-egyutthato amplitudo");
156 title("A becsult es a szenzor altal szolgaltatott Fourier-egyutthatok")
       ;
157 legend("Becsult 1.", "Szenzor 1.", ...
158     "Becsult 2.", "Szenzor 2.", ...
159     "Becsult 3.", "Szenzor 3.", ...
160     "Becsult 4.", "Szenzor 4.", ...
161     "Becsult 5.", "Szenzor 5.", ...
162     "Location", "best");
```

```
grid on;
=======
%% Loading measurement files
measurementFolder = fullfile(fileparts(fileparts(mfilename("fullpath"))
   ), ...
    "Measurements", "MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz");
gwySampFull = load(fullfile(measurementFolder, "pre_gwySampFull.dat"));
sensorDataFull = load(fullfile(measurementFolder, "pre_sensorResonator.
   dat"));
stmpSync = load(fullfile(measurementFolder, "pre_stmpSync.dat"));
signalFreq = load(fullfile(measurementFolder, "signalFreq.dat"));

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
gwySampY = gwySampY - mean(gwySampY);

sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataCoef = sensorDataFull(:, 4:8) + 1i * sensorDataFull(:, 9:13);

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);

%% Synchronizing the sensor timestamps to gateway time
idobelyegPoli = polyfit(stmpSyncSens, stmpSyncGwy, 1);
sensorToGwyTime = polyval(idobelyegPoli, sensorDataTime);

%% Selecting the harmonics measured by the sensor
if sensorDataFull(1, 14) == 0
    harmonics = 1:5;
else
    harmonics = 1:2:9;
end

%% Calculating Fourier coefficients from gateway samples
validPackets = sensorToGwyTime >= gwySampX(1) & ...
    sensorToGwyTime <= gwySampX(end);
packetIndices = find(validPackets);
compareTimes = sensorToGwyTime(validPackets);
sensorCoefficients = sensorDataCoef(validPackets, :);

windowLength = 50;
sampleIndices = interp1(gwySampX, 1:length(gwySampX), ...
```

```
    compareTimes, "previous");
validWindows = sampleIndices >= windowLength;
sampleIndices = sampleIndices(validWindows);
packetIndices = packetIndices(validWindows);
compareTimes = compareTimes(validWindows);
sensorCoefficients = sensorCoefficients(validWindows, :);
gatewayCoefficients = zeros(length(compareTimes), length(harmonics));

for i = 1:length(compareTimes)
    windowIndices = sampleIndices(i) - windowLength + 1:sampleIndices(i
       );
    phase = 2 * pi * signalFreq * ...
        (gwySampX(windowIndices) - gwySampX(1));
    basisFunctions = exp(1i * phase(:) * harmonics);
    fourierMatrix = [2 * real(basisFunctions), -2 * imag(basisFunctions
       )];
    coefficients = fourierMatrix \ gwySampY(windowIndices);
    gatewayCoefficients(i, :) = coefficients(1:length(harmonics)) + ...
        1i * coefficients(length(harmonics) + 1:end);
end

phaseOffset = sensorDataPhi(packetIndices) - ...
    2 * pi * signalFreq * (compareTimes - gwySampX(1));
sensorCoefficients = sensorCoefficients .* exp(1i * phaseOffset(:) *
   harmonics);
coefficientError = sqrt(mean(abs(sensorCoefficients -
   gatewayCoefficients).^2, 1));
disp("Harmonikusonkénti komplex egyutthato RMSE:");
disp("Oszlopok: harmonikus sorszama, komplex egyutthato RMSE");
disp([harmonics(:), coefficientError(:)]);

figure;
for i = 1:length(harmonics)
    subplot(length(harmonics), 1, i);
    plot(compareTimes, real(sensorCoefficients(:, i)));
    hold on;
    plot(compareTimes, real(gatewayCoefficients(:, i)));
    ylabel("Re x_" + harmonics(i));
    grid on;
    if i == 1
        legend("Szenzor (kozos fazis)", "Gateway", "Location", "best");
    end
end
```

```
243 xlabel("Ido [s]");
244 sgtitle("Fourier-egyutthatok valos resze");
245
246 figure;
247 for i = 1:length(harmonics)
248     subplot(length(harmonics), 1, i);
249     plot(compareTimes, imag(sensorCoefficients(:, i)));
250     hold on;
251     plot(compareTimes, imag(gatewayCoefficients(:, i)));
252     ylabel("Im x_" + harmonics(i));
253     grid on;
254     if i == 1
255         legend("Szenzor (kozos fazis)", "Gateway", "Location", "best");
256     end
257 end
258 xlabel("Ido [s]");
259 sgtitle("Fourier-egyutthatok kepzetes resze");
260 >>>>>>> 1ef44c8cf8cf978c4bcb4804ed3445458912562c
```